Making Safe HTML (Native PHP)

Making data safe is not the sole remit of a view, and it would be remiss of me to skip over how to safely work with untrusted data outside of a templating system. Incoming form data might well be dispatched to another system or stored in a database not everything is going to end up inside HTML content. PHP offers a range of built-in functions to detect and/or remove potentially malicious data before it can do any harm inside a web browser.

String Manipulation

Code injection is almost always a matter of injecting one string into another, so it makes sense that PHP has a number of string-handling functions designed to mitigate those risks. I deliberately omitted these functions from Chapter 8, "Strings" (which was already quite huge!), because they would make more sense explained here in the context of all things HTML and HTTP. As we're all nice and fresh from the previous section on all things Twig and templating, these functions should look familiar in their effects. For good reason, too: Twig utilizes some of these functions in its internal implementation (see the `\Twig\Runtime\EscaperRuntime` class). The PHP string functions that deal with markup safety do one of two things: encode meta-characters or remove markup.

Table 11-41. *PHP markup safety functions*

`htmlentities()`	Convert the maximum range of potential characters to HTML entities
`html_entity_decode()`	Reverse of `htmlentities()`
`htmlspecialchars()`	Convert a minimal range of characters
`htmlspecialchars_decode()`	Reverse of `htmlspecialchars()`
`get_html_translation_table()`	Returns array of mappings for `htmlentities()` or `htmlspecialchars()`
`strip_tags()`	Returns string with HTML and PHP tags removed